

CO4-AT2-Industry Problem Solving Task-Questions

Student: JEEVANESHWARAN S

Register Number: 192421331

Q14. Real-Time Stock Trading Strategy Optimization (Dynamic Programming)

Introduction

Real-Time Stock Trading Strategy Optimization is a problem in which a financial system must determine the best sequence of buying and selling decisions to maximize profit. Stock prices fluctuate continuously, and the investor may be restricted by conditions such as a maximum number of transactions, cooldown periods after selling, and the requirement that only one stock can be held at a time.

Dynamic Programming (DP) is suitable for this problem because the optimal solution can be constructed from smaller subproblems. Instead of examining every possible combination of buy and sell decisions, DP stores the best result for each possible state and reuses previously calculated results.

1. Problem Definition

Let the stock price on day i be represented by $\text{price}[i]$.

The objective is to find the maximum possible profit by deciding on every day whether to buy a stock, sell a stock, hold the stock, or do nothing.

The strategy must satisfy all given constraints.

2. Important Constraints

a) Transaction Limit: Suppose the investor is allowed to make at most k transactions. One complete transaction consists of Buy $\rightarrow$ Sell. The algorithm must ensure that completed transactions never exceed k .

b) Cooldown Period: After selling a stock, the investor may be prohibited from buying immediately. For example, with a one-day cooldown: Day 1 Buy, Day 2 Sell, Day 3 Cooldown, Day 4 Buy again.

c) Holding Constraint: The investor cannot buy another stock when already holding one stock.

d) Fluctuating Prices: Stock prices vary from day to day, so the best decision on the current day depends on previous decisions.

e) Profit Maximization: The objective is to obtain maximum profit rather than simply maximizing the number of transactions.

3. Dynamic Programming State Representation

A suitable state representation is $\text{DP}[i][k][s]$, where i is the current day, k is the transaction count, and s represents the stock state.

$s = 0 \rightarrow$ Not holding stock

$s = 1 \rightarrow$ Holding stock

$s = 2 \rightarrow$ Cooldown

Each state represents the maximum profit possible up to a particular day under a specific condition.

4. State Transitions

Not Holding a Stock:

$$\text{DP}[i][k][0] = \max(\text{DP}[i-1][k][0], \text{DP}[i-1][k][1] + \text{price}[i])$$

The first term represents doing nothing, while the second term represents selling the stock.

Holding a Stock:

$$DP[i][k][1] = \max(DP[i-1][k][1], DP[i-1][k-1][0] - \text{price}[i])$$

The first term means continuing to hold the stock, while the second represents buying the stock.

5. Cooldown Transition

For a one-day cooldown after selling, the buy decision on day i should be based on the last valid non-cooldown state:

$$DP[i][1] = \max(DP[i-1][1], DP[i-2][0] - \text{price}[i])$$

The reference to $i-2$ ensures that the cooldown day is respected.

6. Initialization

At the beginning, no stock is held and profit is zero. If the investor buys a stock on the first day, the state becomes negative because money is spent on the purchase.

$$DP[0][0] = 0$$

$$DP[0][1] = -\text{price}[0]$$

7. Backtracking of the Optimal Strategy

After calculating the DP states, the system can identify the maximum profit. To determine the actual trading decisions, the algorithm can store the decision taken at each state.

For example:

Buy on Day 1 → Sell on Day 4 → Cooldown on Day 5 → Buy on Day 6 → Sell on Day 8.

Thus, DP can provide both the maximum profit and the sequence of decisions that generated it.

8. Algorithm

1. Read the stock prices for each day.
2. Set the maximum number of allowed transactions.
3. Initialize the DP states.
4. Process each trading day sequentially.
5. For every transaction count, calculate buy, sell, hold, and cooldown decisions.
6. Select the maximum profit for every state.
7. Apply state-based pruning by retaining only the best result for each state.
8. Continue until the final trading day.
9. Return the maximum achievable profit.
10. Optionally reconstruct the sequence of trading decisions.

9. Pseudocode

```
StockTrading(price, n, k):  
    create DP[n][k+1][2]  
  
    for t = 0 to k:  
        DP[0][t][0] = 0  
        DP[0][t][1] = -price[0]  
  
    for i = 1 to n-1:  
        for t = 1 to k:  
            DP[i][t][0] =  
                max(DP[i-1][t][0],
```

```

        DP[i-1][t-1][1] + price[i])

    DP[i][t][1] =
        max(DP[i-1][t][1],
            DP[i-1][t][0] - price[i])

    return DP[n-1][k][0]

```

10. Example

Consider the stock prices:

[10, 12, 8, 15, 20]

Suppose at most two transactions are allowed.

Transaction 1:

Buy at ■10 and sell at ■12.

Profit = ■12 – ■10 = ■2.

Transaction 2:

Buy at ■8 and sell at ■20.

Profit = ■20 – ■8 = ■12.

Total Profit = ■2 + ■12 = ■14, subject to the transaction and cooldown constraints.

11. Effect of Constraints on Feasible Solutions

Constraints reduce the number of possible trading strategies. A transaction limit prevents excessive buy-sell operations, a cooldown prevents immediate repurchasing, and the holding restriction prevents buying multiple stocks simultaneously.

Dynamic Programming is useful because it avoids repeatedly evaluating equivalent states.

12. Why Dynamic Programming is Suitable

Optimal Substructure: The optimal solution for a larger problem can be obtained from optimal solutions of smaller subproblems.

Overlapping Subproblems: The same state can be reached through different trading sequences. DP stores the best value for that state and reuses it instead of solving it repeatedly.

13. Real-Time Implementation Feasibility

Dynamic Programming can be adapted to a real-time environment because stock prices arrive sequentially. When a new price arrives, the current DP states can be updated using the previous states.

For a fixed transaction limit, only necessary states need to be maintained rather than the complete historical table. This reduces memory use and supports fast updates.

However, practical financial systems must also consider transaction costs, brokerage fees, slippage, volatility, risk limits, execution delay, liquidity, and regulatory restrictions. Therefore, DP is an optimization framework and not a guarantee of future profit.

14. Algorithm Efficiency

For n stock-price observations and at most k transactions:

Time Complexity: $O(nk)$

Space Complexity: $O(nk)$

Because the current day generally depends only on previous states, the space can be optimized to $O(k)$. This is useful in real-time implementations.

15. Advantages

- Finds an optimal strategy under specified constraints.
- Avoids repeated calculations.
- Handles transaction limits and cooldown periods.
- Works with changing stock prices.
- Can reconstruct trading decisions.
- Provides efficient computation for fixed transaction limits.
- Can be adapted for sequential price updates.

16. Limitations

The basic DP model is simplified compared with a real financial environment. Future prices are unknown, and practical systems are affected by transaction costs, sudden market events, slippage, liquidity, network latency, risk limits, and regulatory requirements.

Therefore, DP should be used as a decision-support optimization method rather than as a guaranteed-profit mechanism.

17. Interpretation in Financial Decision-Making

The DP result represents the best achievable profit among the strategies considered under the specified constraints.

For example, if the model produces π_{14} , it means π_{14} is the maximum profit for the given price sequence under the modeled transaction and cooldown rules. A financial system can use this to compare strategies, evaluate constraint effects, and support automated or semi-automated decision-making.

Conclusion

Real-Time Stock Trading Strategy Optimization is a suitable application of Dynamic Programming because it contains overlapping subproblems, optimal substructure, and multiple decision states.

The problem can be modeled using day, transaction count, and holding status. Recurrence relations determine whether to buy, sell, hold, or remain inactive, while cooldown periods and transaction limits are incorporated into state transitions.

The basic DP solution has $O(nk)$ time complexity and can be optimized to $O(k)$ space complexity. This makes it computationally practical for many decision-support applications. However, real-time financial systems must combine the DP model with risk management, transaction costs, market data processing, and execution controls.

Thus, Dynamic Programming provides an effective framework for finding stock trading decisions while satisfying multiple constraints.